

Python E-Course

Module 5 — Data Structures

Detailed Notes

Module Overview

Level: Beginner

Duration: ~2 weeks (12-16 hours)

Prerequisites: Modules 1-4

What You'll Learn

- Use lists, tuples, sets, and dictionaries effectively.
- Choose the right structure for the problem.
- Iterate, mutate, copy, and combine collections.
- Build collections with comprehensions.
- Work with nested structures.

Lessons

#	Lesson	Duration
1	Lists — Basics & Indexing	35 min
2	List Methods & Slicing	35 min
3	Tuples	25 min
4	Sets	30 min
5	Dictionaries	40 min
6	Comprehensions	30 min
7	Nested Structures	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 6: Strings — go deep on Python's text type.

Lesson 1: Lists — Basics & Indexing

Module: 5 — Data Structures

Duration: 35 minutes

Difficulty: Beginner

Learning Objectives

- Create, read, and modify a list.
- Index from the front and the back.
- Iterate over a list with for.
- Use len(), in, and concatenation.

Prerequisites

- Modules 1-4

1. Why This Matters

Lists are the most-used container in Python. Almost every program collects, transforms, or iterates over a list of items — orders, students, log lines, file paths. Master lists and the rest of Python becomes much easier.

2. Concept

2.1 Creating a list

Wrap items in [], separated by commas:

```
nums = [3, 1, 4, 1, 5, 9]
empty = []
mixed = [1, "hi", True, 3.14]  # any types allowed
```

2.2 Indexing

Square brackets with a zero-based index:

```
nums = [10, 20, 30]
print(nums[0])    # 10
print(nums[2])    # 30
```

Negative indexes count from the end:

```
print(nums[-1])   # 30 (last)
print(nums[-2])   # 20
```

Out-of-range index raises IndexError.

2.3 Mutating

Lists are mutable — you can change items in place:

```
nums = [10, 20, 30]
nums[1] = 99
print(nums)    # [10, 99, 30]
```

2.4 Length

```
print(len(nums))    # 3
```

2.5 Membership

```
print(20 in nums)    # True
print(50 not in nums) # True
```

2.6 Concatenation and repetition

```
[1, 2] + [3, 4]    # [1, 2, 3, 4]
[0] * 5             # [0, 0, 0, 0, 0]
```

2.7 Iteration

```
for item in nums:
    print(item)

for index, value in enumerate(nums):
    print(index, value)
```

3. Code Examples

Example 1: Create and index

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
print(fruits[-1])
print(len(fruits))
```

Expected Output:

```
apple
cherry
3
```

Example 2: Mutate

```
nums = [1, 2, 3, 4]
nums[0] = 100
print(nums)
```

Expected Output: [100, 2, 3, 4]

Example 3: Membership and iteration

```
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits)
for f in fruits:
    print("I like", f)
```

Expected Output:

```
True
I like apple
I like banana
I like cherry
```

Example 4: Build a quick list

```
zeros = [0] * 5
ones = [1] * 3
print(zeros + ones)
```

Expected Output: [0, 0, 0, 0, 0, 1, 1, 1]

Example 5: Avoiding IndexError

```
nums = [1, 2, 3]
i = 10
if i < len(nums):
    print(nums[i])
else:
    print("out of range")
```

Expected Output: out of range

4. Common Mistakes

Mistake	What Happens	Fix
nums[len(nums)]	IndexError	Last valid index is len(nums) - 1
Treating string concatenation like list extension	Different operators	list + list or list.append; for strings use +
Modifying a list while iterating	Skipped/duplicated items	Iterate a copy: for x in nums[:]
Using [] thinking it's a tuple	Different type	() for tuple, [] for list

5. Practice Tasks

- Create colors = ["red", "green", "blue"]. Print the first, last, and length.
- Replace the middle item of a list of 5 numbers with 0.
- Iterate over [10, 20, 30, 40] and print index: value using enumerate.

6. Key Takeaways

- Lists store ordered, mutable items in [].
- 0-based indexing; negative indexes count from the end.
- len, in, +, * all work on lists.
- Iterate with for and enumerate.

7. What's Next

List methods (append, pop, sort, ...) and slicing — the tools that make lists powerful.

Lesson 2: List Methods & Slicing

Module: 5 — Data Structures

Duration: 35 minutes

Difficulty: Beginner

Learning Objectives

- Add, remove, sort, and search items with list methods.
- Slice lists with [start:stop:step].
- Distinguish methods that return a value from those that mutate in place.

Prerequisites

- Module 5 Lesson 1

1. Why This Matters

Lists alone are just storage. Methods + slicing turn them into a flexible workhorse: queues, stacks, sorted lists, paginated views — all built from a handful of operations.

2. Concept

2.1 Adding items

Method	What it does
<code>lst.append(x)</code>	Add x to the end
<code>lst.insert(i, x)</code>	Insert x at index i
<code>lst.extend(other)</code>	Add all items from another iterable

```
nums = [1, 2]
nums.append(3)      # [1, 2, 3]
nums.insert(0, 0)   # [0, 1, 2, 3]
nums.extend([4, 5]) # [0, 1, 2, 3, 4, 5]
```

2.2 Removing items

Method	What it does
<code>lst.pop()</code>	Remove and return last item
<code>lst.pop(i)</code>	Remove and return item at i
<code>lst.remove(x)</code>	Remove first occurrence of x
<code>del lst[i]</code>	Delete item at index
<code>lst.clear()</code>	Remove everything

2.3 Sorting and reversing

Method/Function	Behavior
<code>lst.sort()</code>	Sort in place (returns None)
<code>lst.sort(reverse=True)</code>	Descending sort in place
<code>sorted(lst)</code>	Returns a new sorted list
<code>lst.reverse()</code>	Reverse in place

<code>reversed(lst)</code>	Returns a reverse iterator
----------------------------	----------------------------

```

a = [3, 1, 2]
a.sort()           # a is now [1, 2, 3]
b = sorted([3, 1, 2]) # b is [1, 2, 3]; original untouched

```

2.4 Searching and counting

Method	What it does
<code>lst.index(x)</code>	Index of first x (raises <code>ValueError</code> if missing)
<code>lst.count(x)</code>	How many times x appears
<code>min(lst), max(lst), sum(lst)</code>	Built-ins

2.5 Slicing

`lst[start:stop:step]` returns a new list:

```

nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
nums[2:5]      # [2, 3, 4]
nums[:3]       # [0, 1, 2]
nums[7:]       # [7, 8, 9]
nums[::2]      # [0, 2, 4, 6, 8]
nums[::-1]     # reversed copy

```

Stop is exclusive, like range. Negative indexes work too: `nums[-3:]` → last 3.

2.6 Slice assignment

You can assign to a slice:

```

nums = [1, 2, 3, 4]
nums[1:3] = [20, 30, 40]
print(nums)      # [1, 20, 30, 40, 4]

```

2.7 Copying

A naked assignment makes another name for the same list. To copy:

```

a = [1, 2, 3]
b = a           # SAME list
c = a[:]        # copy
d = a.copy()    # copy
e = list(a)     # copy

```

3. Code Examples

Example 1: Append/pop

```

queue = []
queue.append("Alice")
queue.append("Bob")
queue.append("Carol")
served = queue.pop(0)
print(served)
print(queue)

```

Expected Output:

```
Alice  
['Bob', 'Carol']
```

Example 2: Sort

```
scores = [88, 72, 95, 60]  
scores.sort()  
print(scores)  
print(sorted([3, 1, 2], reverse=True))
```

Expected Output:

```
[60, 72, 88, 95]  
[3, 2, 1]
```

Example 3: Slicing

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
print(nums[2:6])  
print(nums[:4])  
print(nums[-3:])  
print(nums[:2])  
print(nums[::-1])
```

Expected Output:

```
[2, 3, 4, 5]  
[0, 1, 2, 3]  
[7, 8, 9]  
[0, 2, 4, 6, 8]  
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Example 4: Aliasing vs copying

```
a = [1, 2, 3]  
b = a  
c = a[:]  
a.append(99)  
print(b)  
print(c)
```

Expected Output:

```
[1, 2, 3, 99]  
[1, 2, 3]
```

Explanation: b is the same list; c is a copy.

Example 5: index() and count()

```
nums = [1, 2, 3, 2, 4, 2]  
print(nums.index(2))  
print(nums.count(2))
```

Expected Output:

1
3

4. Common Mistakes

Mistake	What Happens	Fix
<code>result = nums.sort()</code>	result is None	Use <code>sorted(nums)</code>
<code>lst.index(missing)</code>	<code>ValueError</code>	Check with <code>in</code> first or use <code>try/except</code> (M8)
Aliasing instead of copying	Surprise mutation	Use <code>lst[:]</code> , <code>lst.copy()</code> , or <code>list(lst)</code>
Slicing wrong direction	Empty result	Step must match direction: <code>[5:2:-1]</code>

5. Practice Tasks

- Start with `[]`. Append "a", "b", "c". Then pop the first and print the rest.
- From `nums = list(range(20))`, slice every third number starting at index 1.
- Sort `["banana", "apple", "cherry"]` two ways — alphabetically and by length (use `sorted(..., key=len)`).

6. Key Takeaways

- `Append/extend/insert` add; `pop/remove/del/clear` remove.
- `sort` mutates; `sorted` returns a new list.
- Slicing `[start:stop:step]` returns a new list.
- Assignment aliases; use `[:]` or `.copy()` to actually copy.

7. What's Next

Tuples — immutable cousins of lists, with their own role.

Lesson 3: Tuples

Module: 5 — Data Structures

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Create and use tuples.
- Choose tuple over list for fixed records.
- Use tuple unpacking and named-tuple-like patterns.

Prerequisites

- Module 5 Lesson 2

1. Why This Matters

Tuples are like lists but immutable. That immutability makes them safe as dict keys, safer to pass around (no one can modify them), and the natural shape for fixed records like (latitude, longitude). They also unpack beautifully — you've already used them in Module 4 returns.

2. Concept

2.1 Creating

Wrap items in (), separated by commas:

```
point = (3, 4)
empty = ()
single = (7,)          # comma is required for single-element
```

Parens are optional in many contexts — the comma is what makes a tuple:

```
pair = 1, 2           # also a tuple
```

2.2 Indexing and iteration

Same as lists:

```
point = (3, 4)
print(point[0])
for v in point:
    print(v)
```

2.3 Immutability

```
point = (3, 4)
point[0] = 99      # TypeError
```

No methods that mutate (append, pop, ...) exist on tuples.

2.4 Tuple unpacking

```
x, y = (3, 4)
a, b, c = 1, 2, 3
```

Useful for multiple return values:

```
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([5, 2, 9])
```

2.5 *rest in unpacking

```
first, *rest = [1, 2, 3, 4]
# first = 1, rest = [2, 3, 4]

*init, last = [1, 2, 3, 4]
# init = [1, 2, 3], last = 4
```

2.6 When to use tuple vs list

Tuple	List
Fixed-size record	Growing collection
Heterogeneous data (name, age, city)	Homogeneous items (all scores)
Used as dict key or set element	Used for ordered mutation
Function returns	When you'll add/remove

3. Code Examples

Example 1: Basic tuple

```
point = (3, 4)
print(point)
print(point[0], point[1])
print(len(point))
```

Expected Output:

```
(3, 4)
3 4
2
```

Example 2: Unpacking

```
name, age, city = ("Maya", 22, "Pune")
print(name)
print(age)
print(city)
```

Expected Output:

```
Maya
22
Pune
```

Example 3: Multiple-return + unpacking

```
def min_max(nums):  
    return min(nums), max(nums)  
  
lo, hi = min_max([5, 2, 9, 1, 7])  
print(lo, hi)
```

Expected Output: 1 9

Example 4: Star unpacking

```
first, *middle, last = [1, 2, 3, 4, 5]  
print(first)  
print(middle)  
print(last)
```

Expected Output:

```
1  
[2, 3, 4]  
5
```

Example 5: Immutability

```
t = (1, 2, 3)  
try:  
    t[0] = 99  
except TypeError as e:  
    print("Error:", e)
```

Expected Output: Error: 'tuple' object does not support item assignment

4. Common Mistakes

Mistake	What Happens	Fix
(7) as a one-element tuple	It's just 7 (an int in parens)	Use (7,)
Trying to append to a tuple	AttributeError	Use a list
Unpacking with wrong count	ValueError	Match count or use *rest

5. Practice Tasks

- Create coord = (10, 20) and print each coordinate on its own line using unpacking.
- Write a function returning (min, max, sum) of three numbers; unpack at the call.
- Unpack [1, 2, 3, 4, 5] into head, *tail. Print both.

6. Key Takeaways

- Tuples: immutable, ordered, defined with commas (parens optional).
- Use for fixed records and function returns.
- Unpacking + *rest is concise and Pythonic.
- Can be dict keys or set members; lists cannot.

7. What's Next

Sets — unordered collections with fast membership and set operations.

Lesson 4: Sets

Module: 5 — Data Structures

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Create sets, add and remove items.
- Use set operations: union, intersection, difference.
- Use sets for fast membership checks and de-duplication.

Prerequisites

- Module 5 Lesson 3

1. Why This Matters

Sets store unique items with $O(1)$ membership checks. If you've ever written nested loops to find common items in two lists, a set turns that into a one-liner — and runs orders of magnitude faster on large data.

2. Concept

2.1 Creating

```
colors = {"red", "green", "blue"}
empty = set()           # NOT {} — that's an empty dict
from_list = set([1, 2, 2, 3]) # duplicates removed -> {1, 2, 3}
```

2.2 Properties

- Unordered — no indexing, no slicing.
- Unique — adding a duplicate does nothing.
- Mutable — add, remove, discard, pop, clear.
- Elements must be hashable (no lists, no dicts inside a set).

2.3 Membership and size

```
print("red" in colors)    # True
print(len(colors))        # 3
```

Membership in a set is $O(1)$ — much faster than a list for large data.

2.4 Adding/removing

```
colors.add("yellow")
colors.remove("red")      # KeyError if missing
colors.discard("red")     # no error if missing
colors.pop()              # remove an arbitrary element
```

2.5 Set operations

Operation	Operator	Method	Meaning
Union	<code>`a</code>	<code>b`</code>	<code>a.union(b)</code>
Intersection	<code>a & b</code>	<code>a.intersection(b)</code>	Common
Difference	<code>a - b</code>	<code>a.difference(b)</code>	In a not in b
Symmetric diff	<code>a ^ b</code>	<code>a.symmetric_difference(b)</code>	In one but not both

2.6 De-duplicating a list

```
nums = [1, 2, 2, 3, 1, 4]
unique = list(set(nums))    # order not preserved!
```

For order-preserving unique, use `dict.fromkeys(nums)` (next lesson) or iterate with a seen set.

3. Code Examples

Example 1: Create and add

```
fruits = {"apple", "banana"}
fruits.add("cherry")
fruits.add("apple")          # duplicate ignored
print(fruits)
print(len(fruits))
```

Expected Output (order may vary):

```
{'apple', 'banana', 'cherry'}
3
```

Example 2: Membership speed

```
items = list(range(1000000))
items_set = set(items)
print(999999 in items_set)    # near-instant
print(999999 in items)       # also True but slower (linear scan)
```

Expected Output:

```
True
True
```

Example 3: Union / intersection / difference

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

Expected Output:

```
{1, 2, 3, 4, 5, 6}
{3, 4}
```

```
{1, 2}
{1, 2, 5, 6}
```

Example 4: De-duplication

```
words = ["apple", "banana", "apple", "cherry", "banana"]
unique = set(words)
print(unique)
print(len(unique))
```

Expected Output (order may vary):

```
{'apple', 'banana', 'cherry'}
3
```

Example 5: Find common tags

```
post1_tags = {"python", "tutorial", "beginner"}
post2_tags = {"python", "data", "advanced"}
common = post1_tags & post2_tags
print(common)
```

Expected Output: {'python'}

4. Common Mistakes

Mistake	What Happens	Fix
{ } to make empty set	Creates empty dict	Use set()
Indexing a set: s[0]	TypeError	Sets are unordered
Adding a list as element	TypeError: unhashable type	Use tuple instead
Expecting sorted output	Order is arbitrary	Wrap with sorted(set_)

5. Practice Tasks

- Create a set of your three favorite languages. Add another. Try adding a duplicate.
- Given a = {1, 2, 3} and b = {2, 3, 4}, print union, intersection, and a - b.
- Remove duplicates from [3, 1, 4, 1, 5, 9, 2, 6, 5]. Print the unique count.

6. Key Takeaways

- Sets: unordered, unique, hashable elements.
- O(1) membership — use for fast "is x in this collection".
- Set algebra with |, &, -, ^.
- Use set() (not {}) for an empty set.

7. What's Next

Dictionaries — Python's key/value store, the most useful collection of all.

Lesson 5: Dictionaries

Module: 5 — Data Structures

Duration: 40 minutes

Difficulty: Beginner

Learning Objectives

- Create dictionaries and look up values by key.
- Add, update, and remove entries.
- Iterate over keys, values, and items.
- Use `get()`, `setdefault()`, and dict merging.

Prerequisites

- Module 5 Lesson 4

1. Why This Matters

Dictionaries map keys to values — phone book, settings, JSON, database row. They have $O(1)$ lookup. Once you know dicts, half the code you write becomes dramatically shorter.

2. Concept

2.1 Creating

```
user = {"name": "Maya", "age": 22, "city": "Pune"}
empty = {}
from_pairs = dict([("a", 1), ("b", 2)])
```

Keys must be hashable (strings, numbers, tuples — not lists). Values can be anything.

2.2 Lookup

```
print(user["name"])      # 'Maya'
print(user["unknown"])   # KeyError
```

Safer with `.get()`:

```
print(user.get("unknown"))      # None
print(user.get("unknown", "n/a")) # 'n/a'
```

2.3 Insert / update

```
user["email"] = "maya@example.com" # new key
user["age"] = 23                    # overwrite
```

2.4 Remove

```
del user["city"]
removed = user.pop("age")          # returns value
user.pop("missing", None)          # default avoids KeyError
```

2.5 Membership

```
"name" in user    # True – checks KEYS, not values
```

2.6 Iteration

```
for key in user:
    print(key)

for key, value in user.items():
    print(key, "=", value)

for value in user.values():
    print(value)
```

2.7.setdefault for grouping

```
groups = {}
for word in ["apple", "ant", "bear", "ball"]:
    groups.setdefault(word[0], []).append(word)
# groups -> {'a': ['apple', 'ant'], 'b': ['bear', 'ball']}
```

2.8 Merging

```
a = {"x": 1, "y": 2}
b = {"y": 99, "z": 3}
merged = a | b          # {'x': 1, 'y': 99, 'z': 3}
a.update(b)             # mutates a
```

2.9 Length

```
len(user)    # number of keys
```

3. Code Examples

Example 1: Basic dict

```
user = {"name": "Maya", "age": 22}
print(user["name"])
user["age"] = 23
user["city"] = "Pune"
print(user)
```

Expected Output:

```
Maya
{'name': 'Maya', 'age': 23, 'city': 'Pune'}
```

Example 2: Safe lookup with .get

```
user = {"name": "Maya"}
print(user.get("age"))
print(user.get("age", 0))
```

Expected Output:

```
None
0
```

Example 3: Iterate items

```
prices = {"apple": 50, "bread": 40, "milk": 60}
for item, price in prices.items():
    print(f"{item}: ₹{price}")
```

Expected Output:

```
apple: ₹50
bread: ₹40
milk: ₹60
```

Example 4: Counting word occurrences

```
text = "the rain in spain stays mainly in the plain"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

Expected Output:

```
{'the': 2, 'rain': 1, 'in': 2, 'spain': 1, 'stays': 1, 'mainly': 1, 'plain': 1}
```

Example 5: Group by first letter

```
words = ["apple", "ant", "bear", "ball", "cat"]
groups = {}
for w in words:
    groups.setdefault(w[0], []).append(w)
print(groups)
```

Expected Output: {'a': ['apple', 'ant'], 'b': ['bear', 'ball'], 'c': ['cat']}

Example 6: Merge dicts (Python 3.9+)

```
defaults = {"theme": "light", "lang": "en"}
user = {"lang": "fr"}
config = defaults | user
print(config)
```

Expected Output: {'theme': 'light', 'lang': 'fr'}

4. Common Mistakes

Mistake	What Happens	Fix
Looking up missing key with []	KeyError	Use .get() or check in first
in dict checks values	It checks keys	Use value in dict.values() for values
List as key	TypeError: unhashable	Use tuple
for key, value in d:	ValueError (unpacks key only)	Use d.items()

5. Practice Tasks

- Build {"a": 1, "b": 2, "c": 3}. Add "d": 4. Remove "a". Print result.

- Count letters in "banana" using a dict.
- Given prices = {"apple": 50, "bread": 40}, print "₹50 — apple" for each.

6. Key Takeaways

- Dicts: key → value, O(1) lookup, keys must be hashable.
- Use .get(key, default) to avoid KeyError.
- Iterate with .items(), .keys(), .values().
- .setdefault is great for grouping.

7. What's Next

Comprehensions — build lists, sets, and dicts in a single expressive line.

Lesson 6: Comprehensions

Module: 5 — Data Structures

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Write list, set, and dict comprehensions.
- Use conditional filters inside a comprehension.
- Know when not to use a comprehension.

Prerequisites

- Module 5 Lesson 5

1. Why This Matters

Comprehensions turn 4 lines into 1 — readable, fast, idiomatic. Reading other Pythonists' code means seeing comprehensions everywhere; writing them is a major productivity step.

2. Concept

2.1 List comprehension

```
[expression for item in iterable]
```

```
squares = [n * n for n in range(5)]  
# [0, 1, 4, 9, 16]
```

Equivalent to:

```
squares = []  
for n in range(5):  
    squares.append(n * n)
```

2.2 With a filter

```
[expression for item in iterable if condition]
```

```
evens = [n for n in range(10) if n % 2 == 0]  
# [0, 2, 4, 6, 8]
```

2.3 With if/else expression

The conditional goes before the for when you want to choose between two values:

```
labels = ["even" if n % 2 == 0 else "odd" for n in range(5)]  
# ['even', 'odd', 'even', 'odd', 'even']
```

2.4 Set comprehension

```
{n * n for n in range(5)}  
# {0, 1, 4, 9, 16}
```

2.5 Dict comprehension

```
{key_expr: value_expr for item in iterable}
```

```
sq_map = {n: n * n for n in range(5)}  
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Swap keys and values of an existing dict:

```
inverted = {v: k for k, v in original.items()}
```

2.6 Nested comprehensions

Two fors = nested loops:

```
pairs = [(x, y) for x in range(3) for y in range(3) if x != y]
```

Readable in moderation; if you'd hesitate to read it back next week, write it as nested for loops instead.

2.7 When not to use

- The expression has side effects (printing, logging) — use a regular loop.
- The logic spans 3+ lines — use a regular loop.
- Multiple ifs nested — clearer as a loop with continue.

3. Code Examples

Example 1: Squares

```
print([n * n for n in range(1, 6)])
```

Expected Output: [1, 4, 9, 16, 25]

Example 2: Filter evens

```
print([n for n in range(20) if n % 3 == 0])
```

Expected Output: [0, 3, 6, 9, 12, 15, 18]

Example 3: Conditional expression

```
status = ["adult" if age >= 18 else "minor" for age in [10, 16, 21, 35]]  
print(status)
```

Expected Output: ['minor', 'minor', 'adult', 'adult']

Example 4: Dict comp from two lists

```
names = ["Alice", "Bob", "Carol"]  
ages = [25, 30, 28]  
people = {n: a for n, a in zip(names, ages)}  
print(people)
```

Expected Output: {'Alice': 25, 'Bob': 30, 'Carol': 28}

Explanation: zip pairs elements from two iterables.

Example 5: Set from a string

```
unique_letters = {c for c in "mississippi"}
print(sorted(unique_letters))
```

Expected Output: ['i', 'm', 'p', 's']

Example 6: Nested

```
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [x for row in matrix for x in row]
print(flat)
```

Expected Output: [1, 2, 3, 4, 5, 6]

Explanation: Read left to right: outer loop first, inner second.

4. Common Mistakes

Mistake	What Happens	Fix
Using a comp for side effects	[print(x) for x in xs] builds a useless list	Use a normal for loop
Too-clever nested comps	Unreadable	Refactor to loops or smaller functions
Forgetting if order in if/else	SyntaxError	[A if cond else B for x in xs]

5. Practice Tasks

- Build the list of cubes of 1..10 with a comprehension.
- From [1, 2, 3, 4, 5, 6], build a list of just the evens.
- Create a dict mapping each word in ["hi", "yes"] to its length.

6. Key Takeaways

- [expr for x in xs] is the basic list comp; add if cond to filter.
- Set comp: {expr for ...}; dict comp: {k: v for ...}.
- Use them for concise transforms; avoid for side effects or complex logic.

7. What's Next

The final lesson: nested data structures — lists of dicts, dicts of lists — which is how real-world JSON and tabular data are shaped.

Lesson 7: Nested Structures

Module: 5 — Data Structures

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Build and navigate nested lists and dictionaries.
- Iterate over lists of dictionaries (common in real data).
- Update deeply-nested values safely.

Prerequisites

- Module 5 Lesson 6

1. Why This Matters

Real data is rarely flat. APIs return lists of dicts; CSVs become lists of rows (dicts); configs are dicts of dicts. The patterns in this lesson are what you'll use in Modules 7 (files) and 12 (APIs).

2. Concept

2.1 List of lists

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9],  
]  
print(grid[1][2])    # 6
```

2.2 List of dicts

The shape API responses often take:

```
students = [  
    {"name": "Alice", "score": 88},  
    {"name": "Bob", "score": 72},  
    {"name": "Carol", "score": 95},  
]
```

Iterate with for:

```
for s in students:  
    print(s["name"], s["score"])
```

2.3 Dict of lists

```
courses = {  
    "python": ["Alice", "Bob"],  
    "rust": ["Carol"],  
}  
courses["python"].append("Dan")
```


2.4 Dict of dicts

```
db = {
    "alice": {"age": 30, "city": "Pune"},
    "bob":   {"age": 25, "city": "Delhi"},
}
print(db["alice"]["city"])
db["alice"]["age"] = 31
```

2.5 Safe navigation

If a key might be missing:

```
city = db.get("eve", {}).get("city", "unknown")
```

2.6 Sorting a list of dicts

```
students_sorted = sorted(students, key=lambda s: s["score"], reverse=True)
```

2.7 Comprehensions on nested data

```
top_names = [s["name"] for s in students if s["score"] >= 80]
```

3. Code Examples

Example 1: List of dicts

```
students = [
    {"name": "Alice", "score": 88},
    {"name": "Bob",   "score": 72},
    {"name": "Carol", "score": 95},
]
for s in students:
    print(f"{s['name']}: {s['score']}")
```

Expected Output:

```
Alice: 88
Bob: 72
Carol: 95
```

Example 2: Filter and sort

```
students = [
    {"name": "Alice", "score": 88},
    {"name": "Bob",   "score": 72},
    {"name": "Carol", "score": 95},
    {"name": "Dan",   "score": 65},
]
top = [s for s in students if s["score"] >= 80]
top_sorted = sorted(top, key=lambda s: s["score"], reverse=True)
for s in top_sorted:
    print(s["name"], s["score"])
```

Expected Output:

```
Carol 95
Alice 88
```

Example 3: Dict of lists

```
schedule = {
    "Monday": ["Math", "Science"],
    "Tuesday": ["English", "History"],
    "Wednesday": ["Art"],
}
for day, classes in schedule.items():
    print(f"{day}: {' '.join(classes)}")
```

Expected Output:

```
Monday: Math, Science
Tuesday: English, History
Wednesday: Art
```

Explanation: ", ".join(list) concatenates strings with a separator (Module 6).

Example 4: Dict of dicts — update one entry

```
inventory = {
    "apple": {"qty": 50, "price": 30},
    "banana": {"qty": 80, "price": 10},
}
inventory["apple"]["qty"] -= 10
print(inventory["apple"])
```

Expected Output: {'qty': 40, 'price': 30}

Example 5: Aggregating

```
sales = [
    {"item": "apple", "qty": 3},
    {"item": "banana", "qty": 5},
    {"item": "apple", "qty": 2},
]
totals = {}
for s in sales:
    totals[s["item"]] = totals.get(s["item"], 0) + s["qty"]
print(totals)
```

Expected Output: {'apple': 5, 'banana': 5}

4. Common Mistakes

Mistake	What Happens	Fix
dict[key1][key2] when key1 missing	KeyError	dict.get(key1, {}).get(key2)
Shared inner list [[]] * 3	All three rows share one list	Use a comp: [[] for _ in range(3)]
Forgetting s["..."] quotes	NameError (treats as variable)	Use the string key
Mutating during iteration	RuntimeError	Iterate list(d.items())

5. Practice Tasks

- Build a list of 3 student dicts. Print each.
- Sort that list by score descending.
- Given a list of orders [{"item": ..., "qty": ...}, ...], build a dict of total quantity per item.

6. Key Takeaways

- Combine collections to model real data.
- Index step-by-step: `data[i][key]` or `data[key][nested_key]`.
- Use `.get` chains for safe navigation.
- Sort, filter, and group with `key=` lambdas and comprehensions.

7. What's Next

End of Module 5. Next: Module 6 — Strings, where we go deep on text — methods, slicing, regex.

Module 5 — Exercises

21 exercises (3 per lesson).

Lesson 5.1 — Lists

- (Easy) Create a list of 5 colors; print first, last, and length.
- (Medium) Build a list of integers 1-10. Replace the third item with 999.
- (Hard) Without slicing, print a list of 10 items in reverse order using a for loop.

Lesson 5.2 — Methods/Slicing

- (Easy) Append "end" then pop the first item from ["a", "b", "c"].
- (Medium) Sort [5, 1, 3, 2] ascending and descending. Print both.
- (Hard) From list(range(20)), slice every 4th item starting at index 2.

Lesson 5.3 — Tuples

- (Easy) Create (10, 20, 30). Unpack into a, b, c. Print each.
- (Medium) Write a function returning (sum, product) for two numbers. Unpack at call.
- (Hard) Unpack [1, 2, 3, 4, 5] into first, *middle, last. Print all three.

Lesson 5.4 — Sets

- (Easy) Create a set of three colors; add a fourth; try to add a duplicate.
- (Medium) Find common elements between [1,2,3,4] and [3,4,5,6] using sets.
- (Hard) Count unique words in the sentence "the quick brown fox jumps over the lazy dog the fox".

Lesson 5.5 — Dicts

- (Easy) Build {"a":1, "b":2}; add "c":3; print all keys.
- (Medium) Count letter frequencies in "abracadabra".
- (Hard) Invert {"a":1, "b":2, "c":3} so values become keys.

Lesson 5.6 — Comprehensions

- (Easy) List of squares of 1-10.
- (Medium) List of words from ["apple", "cat", "banana"] longer than 3 characters.
- (Hard) Dict comp mapping numbers 1-5 to their cubes.

Lesson 5.7 — Nested

- (Easy) Build 3 student dicts; print each name and score.
- (Medium) Sort the list of dicts by score descending.
- (Hard) Given [{"item":"a","qty":2}, {"item":"a","qty":3}, {"item":"b","qty":1}], aggregate quantity per item using a dict.

Module 5 — Quiz

10 MCQs.

Q1

What does `[1,2,3] + [4,5]` produce? A. `[1,2,3,4,5]` B. `[5,7,8]` C. error D. `[[1,2,3],[4,5]]`

Answer: A (L1)

Q2

What does `nums.sort()` return? A. sorted list B. None C. reversed list D. `TypeError`

Answer: B — sorts in place. (L2)

Q3

Which is an empty set? A. `{}` B. `[]` C. `set()` D. `()`

Answer: C — `{}` is an empty dict. (L4)

Q4

Which collection has $O(1)$ membership check on average? A. list B. tuple C. set D. string

Answer: C (L4)

Q5

Which is immutable? A. list B. tuple C. dict D. set

Answer: B (L3)

Q6

What does this print?

```
d = {"a": 1}
print(d.get("b", 0))
```

A. `KeyError` B. 0 C. None D. 1

Answer: B — default returned when key missing. (L5)

Q7

Which builds `[0,2,4,6,8]`? A. `[n for n in range(10) if n%2==0]` B. `[n%2 for n in range(10)]` C. `[n for n in range(10) if n%2]` D. `[n*2 for n in range(5)]`

Answer: A (L6); D also produces it incidentally but A is the filter form being asked.

Q8

What is the result of `{1,2,3} & {2,3,4}`? A. `{1,2,3,4}` B. `{2,3}` C. `{1,4}` D. `set()`

Answer: B — intersection. (L4)

Q9

Given `data = {"a": {"b": 5}}`, how do you read 5 safely if "a" might be missing? A. `data["a"]["b"]` B. `data.get("a", {}).get("b")` C. `data.get("a")["b"]` D. `data["a"].get("b", 0)`

Answer: B (L7)

Q10

Which is the right comprehension to map word $\rightarrow$ length for `["hi", "yes"]`? A. `[w: len(w) for w in words]` B. `{w, len(w) for w in words}` C. `{w: len(w) for w in words}` D. `(w: len(w) for w in words)`

Answer: C (L6)

Module 5 — Assignment: Student Gradebook

Estimated time: 1.5-2 hours

Combines: Module 5 + Modules 3-4

Brief

Build a small gradebook that stores students and grades using dicts and lists, then computes summary stats. No classes, no files — just functions over collections.

Requirements

Implement these functions:

- `add_student(book, name)` — add a new student with empty grades list. Returns the updated book (or mutates in place; document your choice).
- `add_grade(book, name, grade)` — append a grade for a student. Raise / print error if student missing.
- `average(book, name)` → float, average for a student. Return None if no grades.
- `top_students(book, n=3)` → list of (name, avg) for the top n by average.
- `class_summary(book)` → returns a dict like `{"highest": ..., "lowest": ..., "mean": ...}` across all students' averages.

Then a `main()` that:

- Initializes an empty book (a dict[str, list[float]]).
- Adds at least 4 students with at least 3 grades each.
- Prints the class summary and the top 2 students.

Constraints

- Use only Modules 1-5 concepts.
- Use dict for book, list for grades, list-of-tuples or list-of-dicts for results.
- Use a comprehension where natural.
- No external libraries.

Expected Behavior

```
== Class Summary ==
Highest avg: 92.0
Lowest avg:  68.0
Class mean:  79.5
```

```
== Top 2 ==
1. Carol  92.0
2. Alice  85.0
```

Grading Rubric

Criterion	Weight
-----------	--------

All 5 functions correct	40%
Uses dicts properly	15%
Uses comprehensions	15%
Summary output	15%
Style + docstrings	15%

Submission

Save as assignment_module5.py.

Module 5 — Mini Project: Inventory Manager

Overview

A CLI inventory manager. Track items, quantities, and prices in a dict. Support add/remove/view/restock/total-value operations.

Concepts Used

- Dicts of dicts (M5 L7)
- List of dict iteration
- Comprehensions for filtering and totals
- Functions (M4)

Features

- Add item with name, quantity, price.
- Remove item by name.
- Show all items in a formatted table.
- Restock: increase quantity of an existing item.
- Compute total inventory value (sum of qty * price).
- Find low-stock items (qty < 5).

Starter Code

```
# Mini Project: Inventory Manager

def add_item(inv, name, qty, price):
    """Add or replace an item in the inventory."""
    inv[name] = {"qty": qty, "price": price}

def remove_item(inv, name):
    """Remove item. No-op if missing."""
    inv.pop(name, None)

def restock(inv, name, qty):
    """Add `qty` to existing item; ignore if missing."""
    if name in inv:
        inv[name]["qty"] += qty

def total_value(inv):
    """Total inventory value (sum of qty * price)."""
    return sum(item["qty"] * item["price"] for item in inv.values())

def low_stock(inv, threshold=5):
    """Return list of (name, qty) pairs for items below threshold."""
    return [(name, info["qty"]) for name, info in inv.items() if info["qty"] <
threshold]

def show(inv):
    """Print inventory as a table."""
```

```

print(f'{"Item":<15}{ "Qty":>6}{ "Price":>10}{ "Value":>10}')
print("-" * 41)
for name, info in inv.items():
    value = info["qty"] * info["price"]
    print(f'{"name":<15}{info["qty"]:>6}{info["price"]:>10.2f}{value:>10.2f}')
print("-" * 41)
print(f'{"TOTAL":<31}{total_value(inv):>10.2f}')

def run():
    inv = {}
    while True:
        cmd = input("\n[add|rem|stock|show|low|total|quit] > ").strip().lower()
        match cmd:
            case "add":
                name = input("Name: ")
                qty = int(input("Qty: "))
                price = float(input("Price: "))
                add_item(inv, name, qty, price)
            case "rem":
                remove_item(inv, input("Name: "))
            case "stock":
                name = input("Name: ")
                qty = int(input("Qty to add: "))
                restock(inv, name, qty)
            case "show":
                show(inv)
            case "low":
                print(low_stock(inv))
            case "total":
                print(f"Total value: {total_value(inv):.2f}")
            case "quit":
                print("Bye!")
                break
            case _:
                print("Unknown command")

if __name__ == "__main__":
    run()

```

Expected Interaction

```

[add|rem|stock|show|low|total|quit] > add
Name: apple
Qty: 50
Price: 30
[add|rem|stock|show|low|total|quit] > add
Name: bread
Qty: 3
Price: 40
[add|rem|stock|show|low|total|quit] > show

```

Item	Qty	Price	Value
apple	50	30.00	1500.00
bread	3	40.00	120.00

```
-----  
TOTAL                                1620.00  
[add|rem|stock|show|low|total|quit] > low  
[('bread', 3)]  
[add|rem|stock|show|low|total|quit] > quit  
Bye!
```

Extension Challenges

- Save/load inventory to a file (preview of Module 7).
- Add categories: group items by category field.
- Add a sort command to display sorted by name, qty, or value.

Grading Rubric

Criterion	Weight
All commands work	35%
Formatted table	20%
Comprehensions used	15%
Functions are small + named clearly	15%
Style	15%